Day4:
Sorting and Algorithm Analysis

Asst. Prof. Dr. Aurawan Imsombut

https://kasets.art/apRjPx

MODIFY FROM LEVITIN "INTRODUCTION TO THE DESIGN & ANALYSIS OF ALGORITHMS," 3RD ED., CH. 3, 4, 5, 6 ©2012 PEARSON EDUCATION, INC. UPPER SADDLE RIVER, NJ. ALL RIGHTS RESERVED.

Outline

- Algorithm Analysis
  - Asymptotic analysis
  - Order of growth
  - Best-case, average-case, worst-case

Outline

- Sorting
  - Selection sort
  - Bubble sort

    Brute Force Algorithm
  - Insertion sort
  - Shell sort

    Decrease and Conquer Algorithm
  - Merge sort
  - Quick sort

    Divide and Conquer Algorithm
  - Heap sort

    Transform and Conquer Algorithm

Algorithm Analysis

Asymptotic analysis

- Issues:
  - correctness
  - time efficiency *
  - space efficiency
  - optimality

- Approaches:
  - theoretical analysis
  - empirical analysis

Theoretical analysis of time efficiency

Time efficiency is analyzed by determining the number of repetitions of the *basic operation* as a function of *input size*

- *Basic operation*: the operation that contributes most towards the running time of the algorithm

input size

$$T(n) \approx c_{op} C(n)$$

running time

execution time
for basic operation

Number of times
basic operation is executed

Theoretical analysis of time efficiency

```
for (i = 1; i <= n; i++)
    for(j = 1; j <= n; j++)
        sum += j;
```

basic operation

$$\sum_{i=1}^{n}\sum_{j=1}^{n} 1 = \sum_{i=1}^{n} n = \boxed{n^2}$$

```
for (i = 1; i <= n; i++)
    for(j = i+1; j <= n; j++)
        sum += cosine(d[i][j])*i;
```

Which one is faster?

$$\sum_{i=1}^{n}\sum_{j=i+1}^{n} 1 = \sum_{i=1}^{n}(n-i) = \sum_{i=1}^{n} n - \sum_{i=1}^{n} i = n^2 - \frac{n(n+1)}{2} = \boxed{\frac{n^2}{2} - \frac{n}{2}}$$

Order of growth

n	n^2	Order of growth	$\frac{n^2}{2} - \frac{n}{2}$	Order of growth
10	100		45	
20	400	4	190	4.22
40	1,600	4	780	4.11
80	6,400	4	3,160	4.05
160	25,600	4	12,720	4.03
320	102,400	4	51,040	4.01
640	409,600	4	204,480	4.01
1,280	1,638,400	4	818,560	4.00

เพิ่มขึ้นทีละ 2 เท่า

เพิ่มขึ้นทีละ 4 เท่า

เพิ่มขึ้นทีละ 4 เท่า

They have the same order of growth

Order of growth

n	$10n$	Order of growth	$\frac{n^2}{1000}$	Order of growth
10	100		1	
20	200	2	4	4
40	400	2	16	4
80	800	2	64	4
160	1600	2	256	4
320	3200	2	1024	4
640	6400	2	4096	4
1280	12800	2	16384	4
2560	25600	2	65536	4

เพิ่มขึ้นทีละ 2 เท่า

เพิ่มขึ้นทีละ 2 เท่า

เพิ่มขึ้นทีละ 4 เท่า

Order of growth $n^2 > n$

Order of growth

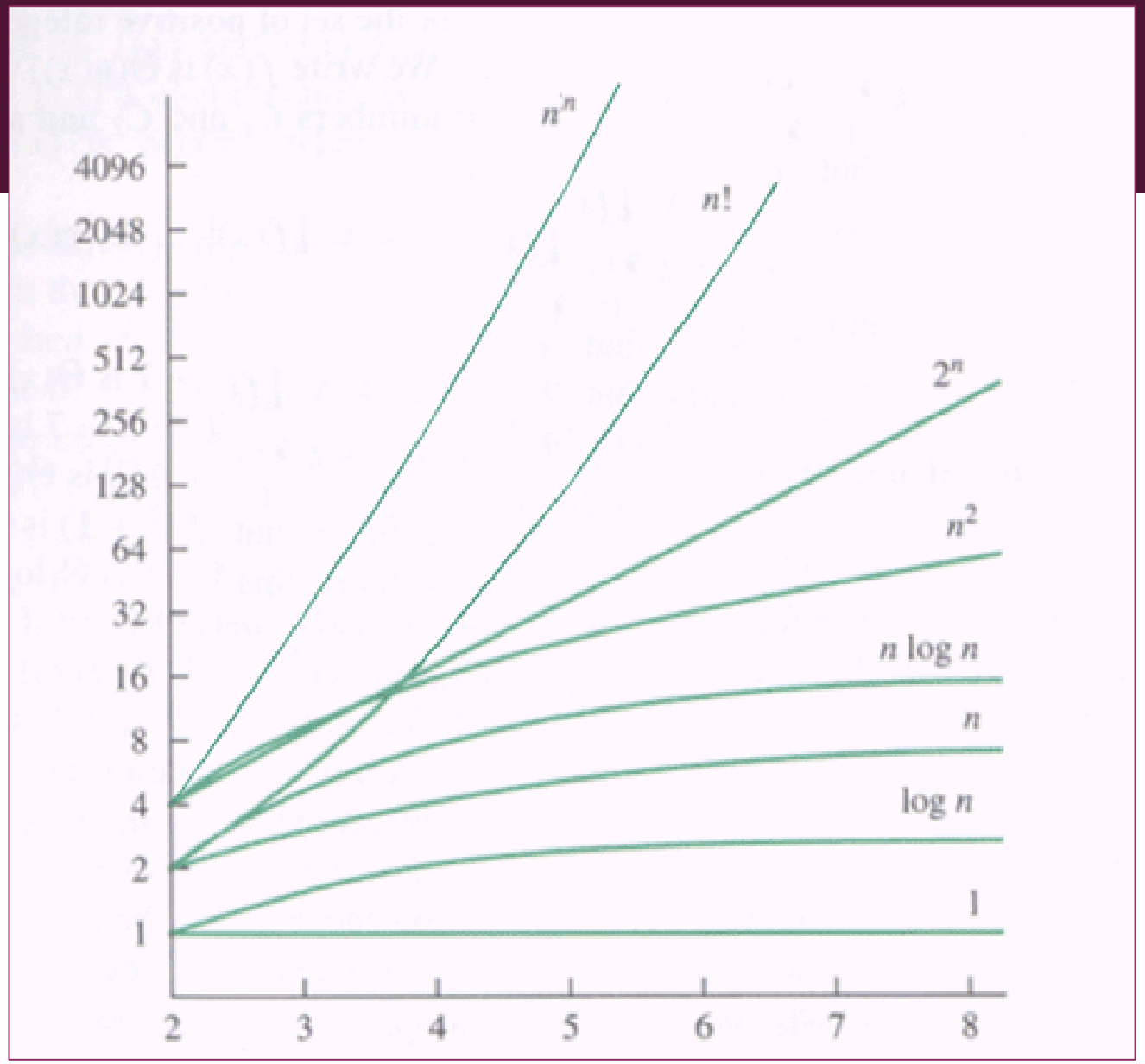
- Most important: Order of growth within a constant multiple as $n \to \infty$

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.3	10^1	$3.3 \cdot 10^1$	10^2	10^3	10^3	$3.6 \cdot 10^6$
10^2	6.6	10^2	$6.6 \cdot 10^2$	10^4	10^6	$1.3 \cdot 10^{30}$	$9.3 \cdot 10^{157}$
10^3	10	10^3	$1.0 \cdot 10^4$	10^6	10^9		
10^4	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Table 2.1 Values (some approximate) of several functions important for analysis of algorithms

Order of growth

- n!: factorial
- a^n: exponential
- n^a: polynomial
- n^2: quadratic
- n^3: cubic
- n: linear
- $\log_a n$: logarithmic
- c: constant



Best-case, average-case, worst-case

For some algorithms efficiency depends on form of input:

- Worst case: $C_{worst}(n)$ – maximum over inputs of size n
- Best case: $C_{best}(n)$ – minimum over inputs of size n
- Average case: $C_{avg}(n)$ – "average" over inputs of size n
  - Number of times the basic operation will be executed on typical input
  - NOT the average of worst and best case
  - Expected number of basic operations considered as a random variable under some assumption about the probability distribution of all possible inputs

Example: Sequential search

ALGORITHM *SequentialSearch*$(A[0..n-1], K)$

```
//Searches for a given value in a given array by sequential search
//Input: An array A[0..n − 1] and a search key K
//Output: The index of the first element of A that matches K
//        or −1 if there are no matching elements
i ← 0
while i < n and A[i] ≠ K do
    i ← i + 1
if i < n return i
else return −1
```

- Worst case
- Best case
- Average case

Average-Case Analysis of Sequential Search

- If the element that we search for always exists in the list, then $C_{avg}(n) = \frac{(n+1)}{2}$.

  That is, on average, we visit half of the entries in the list to search for any element in the list.

  Average case time = $\frac{all\ posible\ case\ time}{no.of\ cases}$

  Average case time = $\frac{1+2+3+\cdots+n}{n} = \frac{\frac{n(n+1)}{2}}{n} = \frac{n+1}{2}$

- If all the time, the element that we will search never exists), then $C_{avg}(n) = n$.

  That is, we visit all the elements in the list.

Asymptotic order of growth

- A way of comparing functions that ignores constant factors and small input sizes
- *O(g(n)):* Big-oh, class of functions *f(n)* that grow no faster than *g(n)*

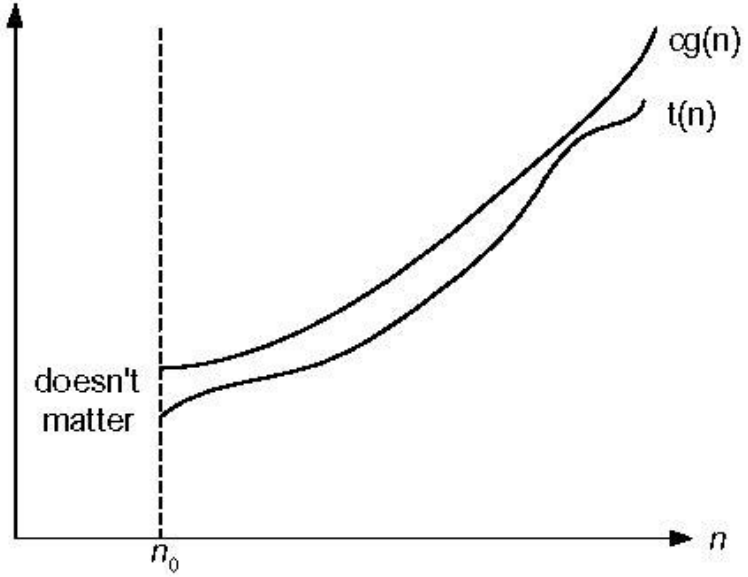


Figure 2.1 Big-oh notation: $t(n) \in O(g(n))$

Example 1: Maximum element

ALGORITHM *MaxElement*$(A[0..n-1])$

```
//Determines the value of the largest element in a given array
//Input: An array A[0..n − 1] of real numbers
//Output: The value of the largest element in A
maxval ← A[0]
for i ← 1 to n − 1 do
    if A[i] > maxval
        maxval ← A[i]
return maxval
```

basic operation

$$C(n) = \sum_{i=1}^{n-1} 1 = n - 1 \in \Theta(n)$$

worst case = average case = best case

C(n): the number of times the comparison

Sorting

Selection Sort

- Scan the array to find its smallest element and swap it with the first element.
- Then, starting with the second element, scan the elements to the right of it to find the smallest among them and swap it with the second elements.
- Generally, on pass `i` `(0 ≤ i ≤ n-2)`, find the smallest element in `A[i..n-1]` and swap it with `A[i]`:

```
A[0] ≤ ... ≤ A[i-1] | A[i], ... , A[min], ..., A[n-1]
```

in their final positions

Selection Sort Example

89	45	68	90	29	34	17	scan 0-6, smallest 17, swap 89 and 17
17	45	68	90	29	34	89	scan 1-6, smallest 29, swap 45 and 29
17	29	68	90	45	34	89	scan 2-6, smallest 34, swap 68 and 34
17	29	34	90	45	68	89	scan 3-6, smallest 45, swap 90 and 45
17	29	34	45	90	68	89	scan 4-6, smallest 68, swap 90 and 68
17	29	34	45	68	90	89	scan 5-6, smallest 89, swap 90 and 89
17	29	34	45	68	89	90	done

Analysis of Selection Sort

```
ALGORITHM  SelectionSort(A[0..n − 1])
    //Sorts a given array by selection sort
    //Input: An array A[0..n − 1] of orderable elements
    //Output: Array A[0..n − 1] sorted in ascending order
    for i ← 0 to n − 2 do
        min ← i
        for j ← i + 1 to n − 1 do
            if A[j] < A[min]   min ← j
        swap A[i] and A[min]
```

The input size is given by the number of elements n

The basic operation is the key comparison

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1 = \sum_{i=0}^{n-2} [(n-1) - (i+1) + 1] = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2)$$

$C(n)$: The number of key comparisons

However, The number of key swaps is only $\Theta(n)$.

Bubble Sort

- Compare adjacent elements of the list and exchange them if they are out of order.
- By doing it repeatedly, we end up "bubbling up" the largest element to the last position on the list.
- The next pass bubbles up the second largest element, and so on, until after `n - 1` passes the list is sorted.

$$A_0, \ldots, A_j \overset{?}{\leftrightarrow} A_{j+1}, \ldots, A_{n-i-1} \mid \underbrace{A_{n-i} \leq \cdots \leq A_{n-1}}_{\text{in their final positions}}$$

Bubble Sort Example

89	45	68	90	29	34	17
45	89	68	90	29	34	17
45	68	89	90	29	34	17
45	68	89	29	90	34	17
45	68	89	29	34	90	17
45	68	89	29	34	17	90

45	68	89	29	34	17	90
45	68	29	89	34	17	90
45	68	29	34	89	17	90
45	68	29	34	17	89	90

etc.

Analysis of Bubble Sort

ALGORITHM *BubbleSort*$(A[0..n-1])$

```
//Sorts a given array by bubble sort
//Input: An array A[0..n − 1] of orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
for i ← 0 to n − 2 do
    for j ← 0 to n − 2 − i do
        if A[j + 1] < A[j] swap A[j] and A[j + 1]
```

$$C(n) = \sum_{i=0}^{n-2} \sum_{j=0}^{n-2-i} 1 = \sum_{i=0}^{n-2} [(n-2-i) - 0 + 1] = \sum_{i=0}^{n-2} (n-1-i) = \frac{(n-1)n}{2} \in \Theta(n^2)$$

$C(n)$: The number of key comparisons

The number of key swaps depends on the input.

$$S_{worst}(n) = C(n) = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Brute Force Sorting Algorithm

- Selection sort and Bubble sort are **Brute Force** sorting algorithm.
- A straightforward approach, usually based directly on the problem's statement and definitions of the concepts involved
- Examples:
  - Computing a^n ($a > 0$, n a nonnegative integer)
  - Computing n!
  - Multiplying two matrices
  - Searching for a key of a given value in a list

Insertion Sort

- Based on technique of card players to arrange a hand
  - Player keeps cards picked up so far in *sorted order*
  - When the player picks up a new card
    - Makes room for the new card
    - Then inserts it in its proper place

Insertion Sort

- To sort array `A[0..n-1]`, sort `A[0..n-2]` recursively and then insert `A[n-1]` in its proper place among the sorted `A[0..n-2]`

$$A[0] \le \cdots \le A[j] < A[j+1] \le \cdots \le A[i-1] \mid A[i] \cdots A[n-1]$$

smaller than or equal to $A[i]$ greater than $A[i]$

- Usually implemented bottom up (non-recursively)

- Example: Sort `6, 4, 1, 8, 5`

```
6 | 4     1     8     5
4   6 |   1     8     5
1   4     6 |   8     5
1   4     6     8 |   5
1   4     5     6     8
```

Pseudocode of Insertion Sort

ALGORITHM *InsertionSort*$(A[0..n-1])$

```
//Sorts a given array by insertion sort
//Input: An array A[0..n − 1] of n orderable elements
//Output: Array A[0..n − 1] sorted in nondecreasing order
for i ← 1 to n − 1 do
    v ← A[i]
    j ← i − 1
    while j ≥ 0 and A[j] > v do
        A[j + 1] ← A[j]
        j ← j − 1
    A[j + 1] ← v
```

Analysis of Insertion Sort

Time efficiency $C(n)$: The number of key comparisons

$$C_{worst}(n) = \sum_{i=1}^{n-1} \sum_{j=0}^{i-1} 1 = \sum_{i=1}^{n-1} i = \frac{(n-1)n}{2} \in \boldsymbol{\Theta(n^2)}$$

$$C_{best}(n) = \sum_{i=1}^{n-1} 1 = (n-1) \in \boldsymbol{\Theta(n)}$$ (also fast on almost sorted arrays)

$$C_{avg}(n) \approx \frac{n^2}{4} \in \boldsymbol{\Theta(n^2)}$$

2. $\sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2)$

Comparison of Quadratic Sorts

- Efficiency is $\Theta(n^2)$, hence called a *quadratic* sort

- None good for large arrays!

Comparison of Quadratic Sorts

	Number of Comparisons		Number of Exchanges	
	Best	Worst	Best	Worst
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n)$
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(1)$	$\Theta(n^2)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n)$	$\Theta(n^2)$

Comparison of order of growth

n	n^2	nlogn
8	64	24
16	256	64
32	1,024	160
64	4,096	384
128	16,384	896
256	65,536	2,047

Shell Sort: A Better Insertion Sort

- Shell sort is a variant of insertion sort
  - It is named after Donald Shell
  - Average performance: $\Theta(n^{3/2})$ or better
- Divide and conquer approach to insertion sort
  - Sort many smaller subarrays using insertion sort
  - Sort progressively larger arrays
  - Finally sort the entire array
- These arrays are elements separated by a *gap*
  - Start with *large gap*
  - Decrease the gap on each "pass"

Shell Sort Example

Gap = 5

12	4	3	9	18	7	2	17	13	1	5	6

Sort every 5th element:

Gap = 3

5	2	3	9	1	7	4	17	13	18	12	6

Sort every 3rd element:

Gap = 1

4	1	3	5	2	6	9	12	7	18	17	13

Final a normal insertion sort:

1	2	3	4	5	6	7	9	12	13	17	18

https://riptutorial.com/algorithm/example/24649/shell-sort-basic-information

Shell Sort Algorithm

- Step 1 – Initialize the value of gap
- Step 2 – Divide the list into smaller sub-list of equal interval gap
- Step 3 – Sort these sub-lists using insertion sort
- Step 4 – Repeat until complete list is sorted

Analysis of Shell Sort

- ***Intuition:*** Reduces work by moving elements farther earlier
- Its general analysis is an open research problem
- Performance depends on sequence of gap values
  - For sequence 2^k, performance is $\boldsymbol{\Theta(n^2)}$ (Gap: $\left\lfloor \frac{N}{2} \right\rfloor, \left\lfloor \frac{N}{4} \right\rfloor, \ldots, 1$)
  - Hibbard's sequence $(2^k - 1)$, performance is $\boldsymbol{\Theta(n^{3/2})}$
    (Gap: …, 63, 31, 15, 7, 3, 1)
  - Successive numbers of the form $2^p 3^q$ (3-smooth numbers), performance is $\boldsymbol{\Theta(n \log n)}$
    (Gap: …, 12, 9, 8, 6, 4, 3, 2, 1)

https://en.wikipedia.org/wiki/Shellsort

Merge sort

- Split array `A[0..n-1]` in two about equal halves and make copies of each half in arrays B and C
- Sort arrays B and C recursively
- Merge sorted arrays B and C into array A as follows:
  - Repeat the following until no elements remain in one of the arrays:
    - compare the first elements in the remaining unprocessed portions of the arrays
    - copy the smaller of the two into A, while incrementing the index indicating the unprocessed portion of that array
  - Once all elements in one of the arrays are processed, copy the remaining unprocessed elements from the other array into A.

Pseudocode of Merge sort

ALGORITHM *Mergesort*$(A[0..n-1])$

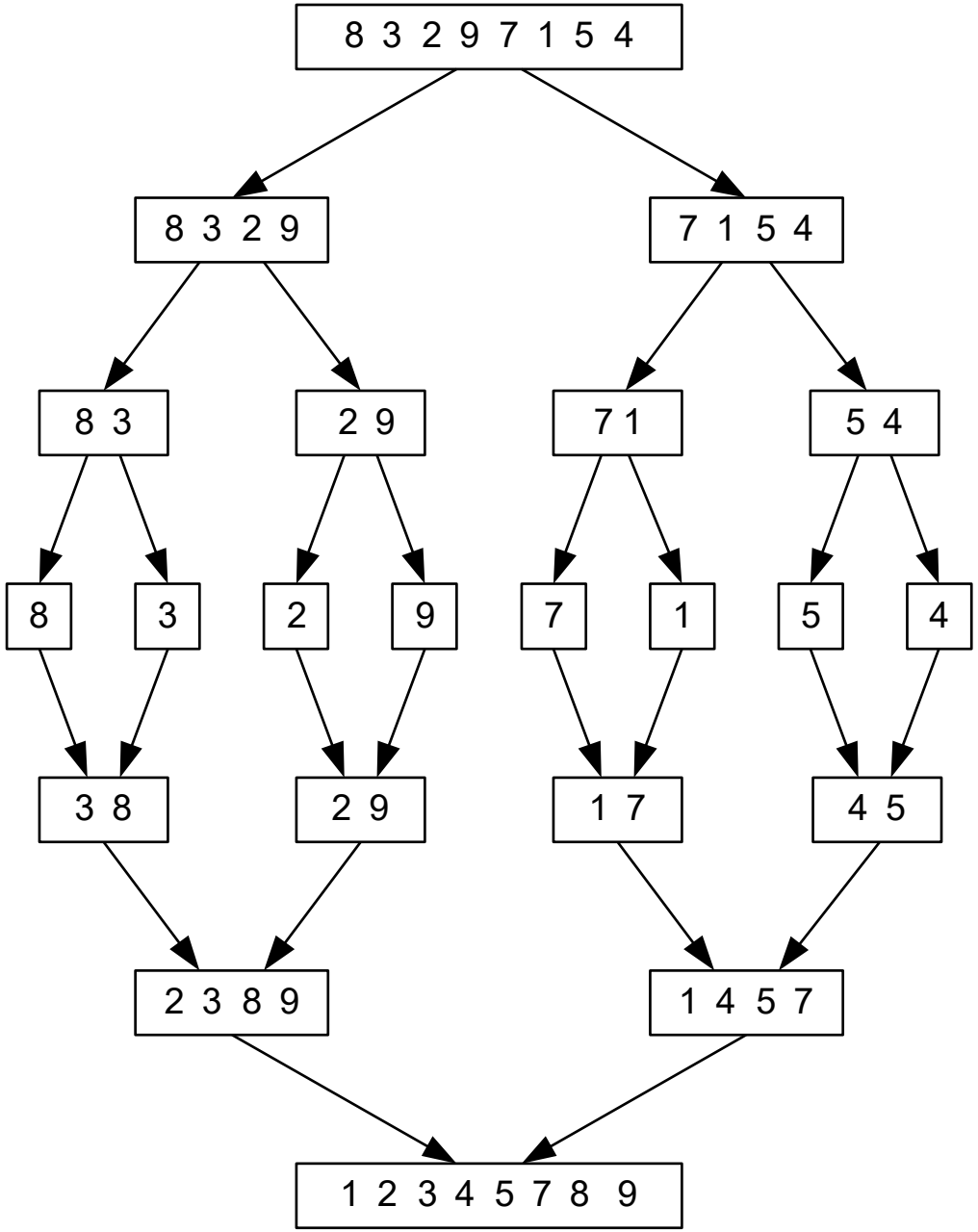
//Sorts array $A[0..n-1]$ by recursive mergesort
//Input: An array $A[0..n-1]$ of orderable elements
//Output: Array $A[0..n-1]$ sorted in nondecreasing order
if $n > 1$
copy $A[0..\lfloor n/2 \rfloor - 1]$ to $B[0..\lfloor n/2 \rfloor - 1]$
copy $A[\lfloor n/2 \rfloor..n-1]$ to $C[0..\lceil n/2 \rceil - 1]$
Mergesort$(B[0..\lfloor n/2 \rfloor - 1])$
Mergesort$(C[0..\lceil n/2 \rceil - 1])$
Merge(B, C, A)

Pseudocode of Merge

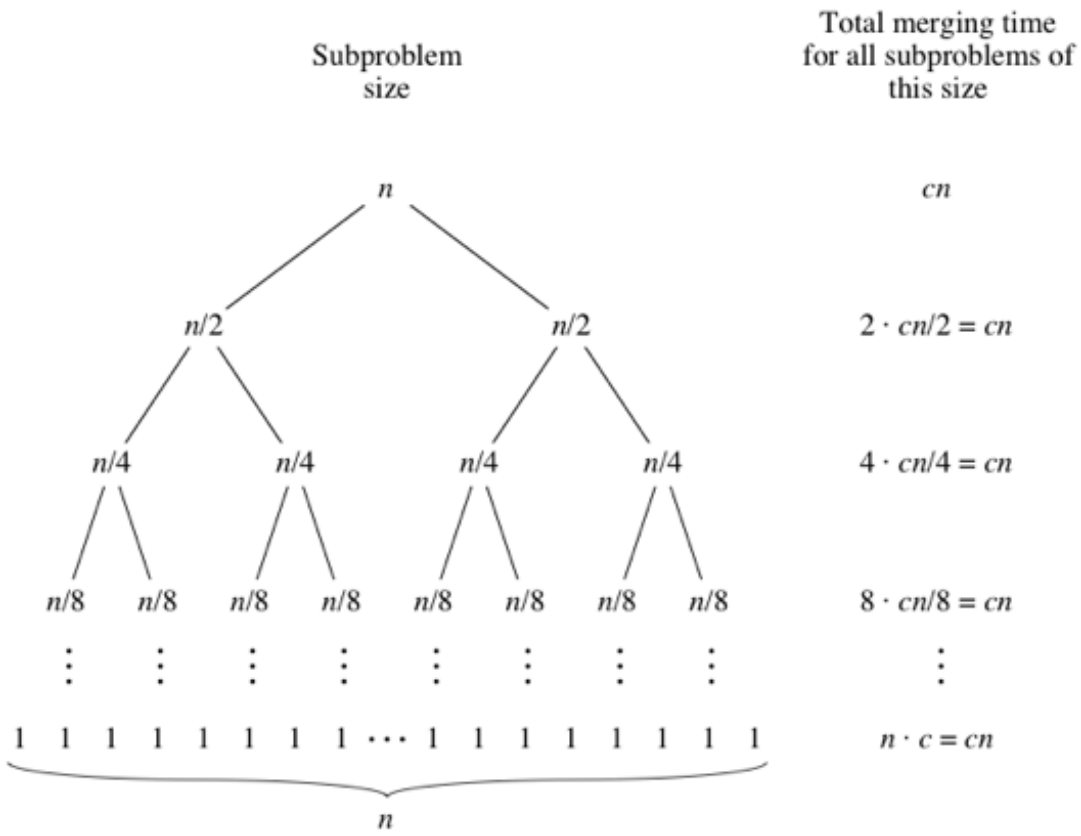
ALGORITHM $Merge(B[0..p-1], C[0..q-1], A[0..p+q-1])$

```
//Merges two sorted arrays into one sorted array
//Input: Arrays B[0..p − 1] and C[0..q − 1] both sorted
//Output: Sorted array A[0..p + q − 1] of the elements of B and C
i ← 0; j ← 0; k ← 0
while i < p and j < q do
    if B[i] ≤ C[j]
        A[k] ← B[i]; i ← i + 1
    else A[k] ← C[j]; j ← j + 1
    k ← k + 1
if i = p
    copy C[j..q − 1] to A[k..p + q − 1]
else copy B[i..p − 1] to A[k..p + q − 1]
```

Merge sort Example



Analysis of Merge sort



c: the constant time of the divide and combine steps

- If there are l levels in the tree, then the total merging sort time is $l \cdot cn$
- when we analyzed binary search, and the answer is
  $l = \log_2 n + 1$
- The total time for mergesort is
  $cn(\log_2 n + 1)$
  $\in \boldsymbol{\Theta(n \log n)}$

https://www.khanacademy.org/computing/computer-science/algorithms/merge-sort/a/analysis-of-merge-sort

Analysis of Merge sort

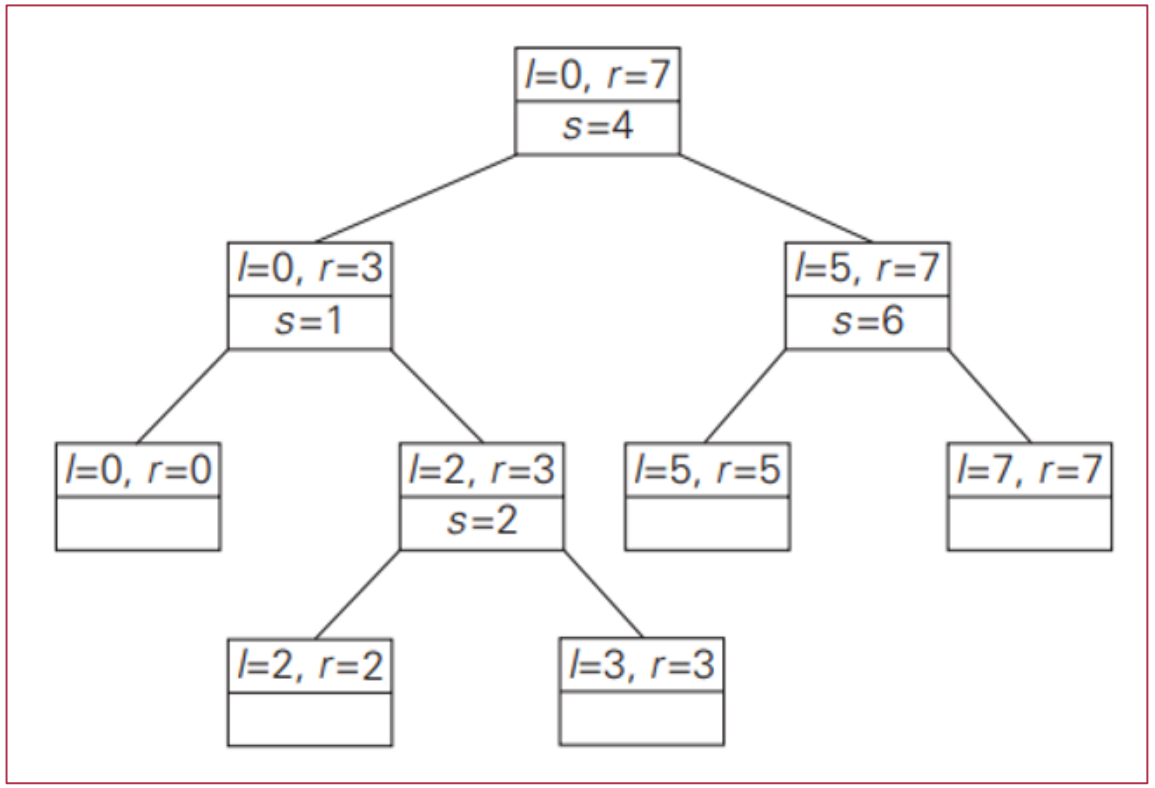
- All cases have same efficiency: $\Theta(n \log n)$
- Space requirement: $\Theta(n)$ (<u>not</u> in-place)
- Can be implemented without recursion (bottom-up)

Quicksort

- Quicksort is the other important sorting algorithm, based on the divide-and-conquer approach.
- Select a pivot (A[s]: partitioning element) – here, the first element
- Rearrange the list so that all the elements in the first s positions are smaller than or equal to the pivot and all the elements in the remaining n-s positions are larger than or equal to the pivot
- $$\underbrace{A[0]\ldots A[s-1]}_{\text{all are } \leq A[s]} \quad A[s] \quad \underbrace{A[s+1]\ldots A[n-1]}_{\text{all are } \geq A[s]}$$
- Exchange the pivot with the last element in the first (i.e., ≤) subarray — the pivot is now in its final position
- Sort the two subarrays recursively

Quicksort Example

0	1	2	3	4	5	6	7
	i						*j*
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	9	8	2	4	7
			i			*j*	
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	8	2	9	7
				i	*j*		
5	3	1	4	2	8	9	7
				j	*i*		
5	3	1	4	2	8	9	7
2	3	1	4	**5**	8	9	7
	i		*j*				
2	3	1	4				
	i	*j*					
2	3	1	4				
	i	*j*					
2	1	3	4				
	j	*i*					
2	1	3	4				
1	**2**	3	4				
1							
			i j				
		3	4				
		j	*i*				
		3	4				
			4				
						i	*j*
					8	9	7
						i	*j*
					8	7	9
						j	*i*
					8	7	9
					7	**8**	9
					7		
							9



l:left, r: right, s: split position

Pseudocode of Quicksort

ALGORITHM *Quicksort*$(A[l..r])$

```
//Sorts a subarray by quicksort
//Input: Subarray of array A[0..n − 1], defined by its left and right
//        indices l and r
//Output: Subarray A[l..r] sorted in nondecreasing order
if l < r
    s ← Partition(A[l..r])  //s is a split position
    Quicksort(A[l..s − 1])
    Quicksort(A[s + 1..r])
```

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n)$$

Hoare's Partitioning Algorithm

ALGORITHM *HoarePartition*$(A[l..r])$

```
//Partitions a subarray by Hoare's algorithm, using the first element
//          as a pivot
//Input: Subarray of array A[0..n − 1], defined by its left and right
//          indices l and r (l < r)
//Output: Partition of A[l..r], with the split position returned as
//          this function's value
p ← A[l]
i ← l; j ← r + 1
repeat
    repeat i ← i + 1 until A[i] ≥ p
    repeat j ← j − 1 until A[j] ≤ p
    swap(A[i], A[j])
until i ≥ j
swap(A[i], A[j])    //undo last swap when i ≥ j
swap(A[l], A[j])
return j
```

Analysis of Quicksort

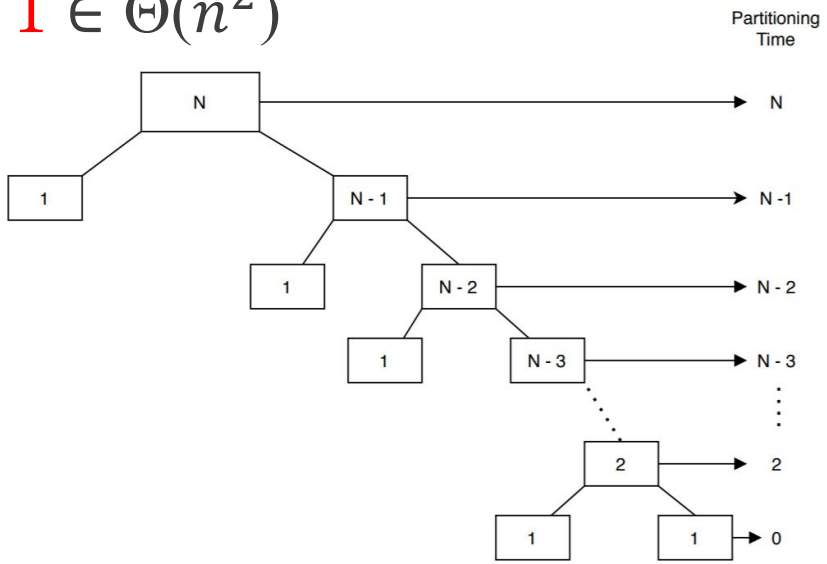
- Best case: split in the middle — $\Theta(\, n \log n \,)$

$$c_{best}(n) = 2c_{best}\left(\frac{n}{2}\right) + n \in \Theta(\, n \log n \,)$$

- Worst case: sorted array! — $\Theta(n^2)$

$$c_{worst}(n) = n + (n-1) + \cdots + 2 = \frac{n(n+1)}{2} - 1 \in \Theta(n^2)$$

- Average case: random arrays — $\Theta(\, n \log n \,)$



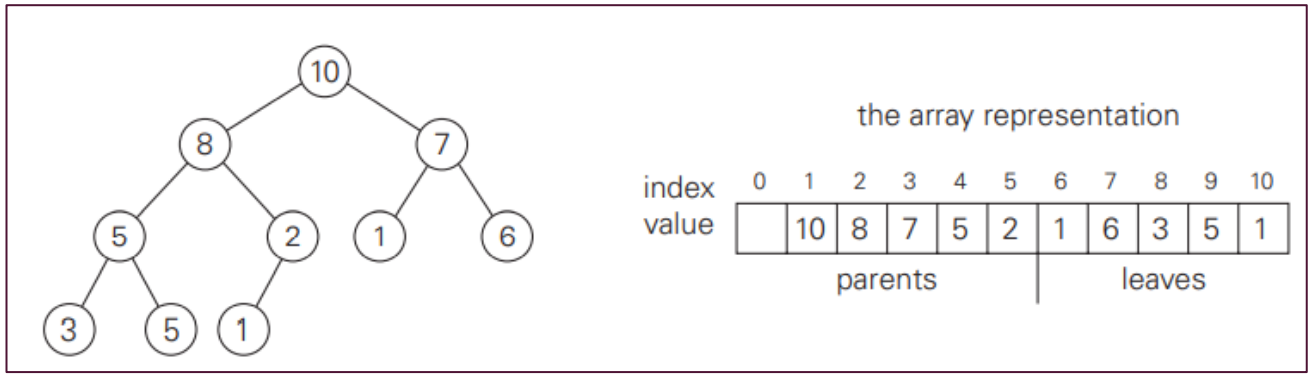
Analysis of Quicksort

- Considered the method of choice for internal sorting of large files ($n \geq 10000$)
- Improvements:
  - better pivot selection: median of three partitioning
  - switch to insertion sort on small subfiles
  - elimination of recursion
- These combine to 20-25% improvement

Heapsort

<u>Definition</u> A *heap* is a binary tree with keys at its nodes (one key per node) such that:

- It is essentially complete, i.e., all its levels are full except possibly the last level, where only some rightmost keys may be missing



- The key at each node is ≥ keys at its children

Heapsort

Stage 1: Construct a heap for a given list of *n* keys

Stage 2: Repeat operation of root removal *n*-1 times:

- Exchange keys in the root and in the last (rightmost) leaf
- Decrease heap size by *1*
- If necessary, swap new root with larger child until the heap condition holds

Example of Sorting by Heapsort

Sort the list 2, 9, 7, 6, 5, 8 by heapsort

Stage 1 (heap construction)

2 9 7 6 5 8

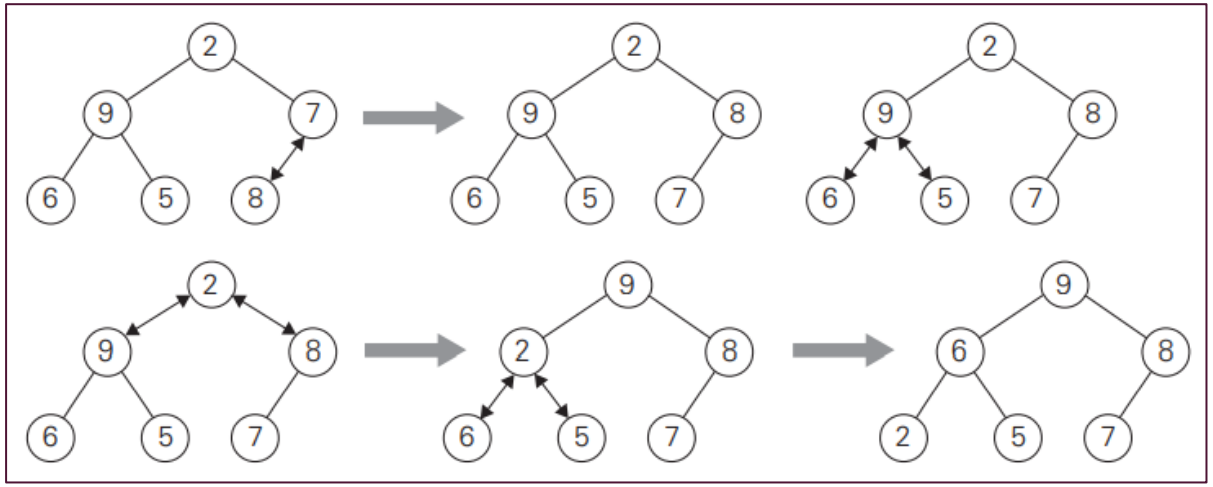
2 9 8 6 5 7

2 9 8 6 5 7

9 2 8 6 5 7

9 6 8 2 5 7

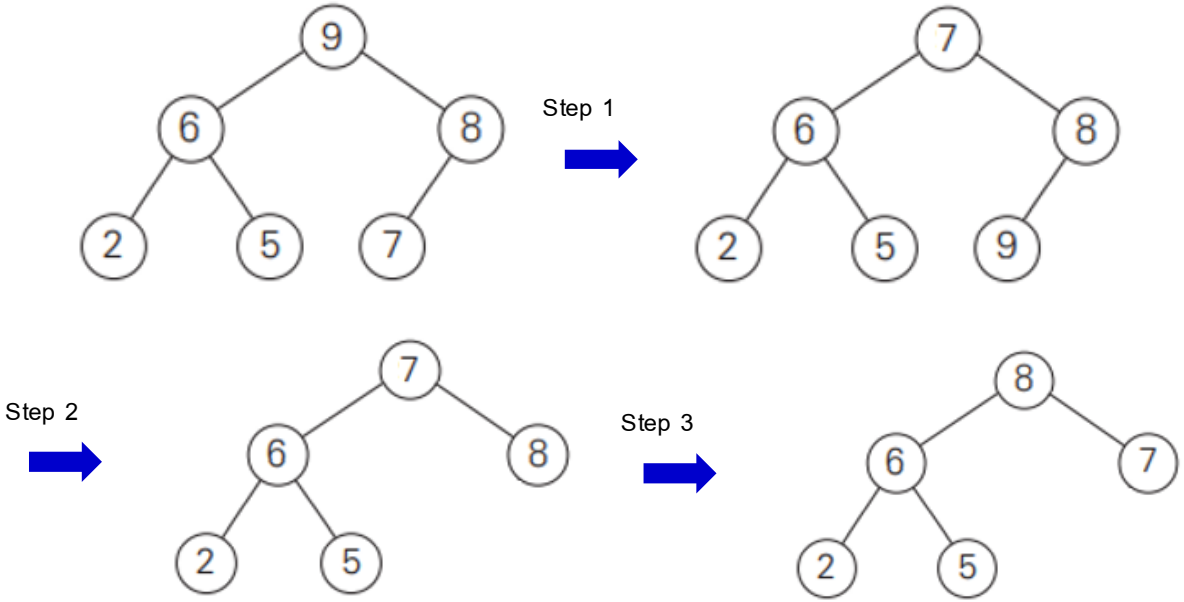
Bottom-up construction of a heap



Example of Sorting by Heapsort

- Example of deleting the root's key from a heap.

  The key to be deleted is swapped with the last key after which the smaller tree is "heapified" by exchanging the new key in its root with the larger key in its children until the parental dominance requirement is satisfied.



Sort the list 2, 9, 7, 6, 5, 8 by heapsort

Stage 2 (root/max removal)

<u>9</u> 6 8 2 5 7
7 6 8 2 5 | 9
<u>8</u> 6 7 2 5 | 9
5 6 7 2 | 8 9
<u>7</u> 6 5 2 | 8 9
2 6 5 | 7 8 9
<u>6</u> 2 5 | 7 8 9
5 2 | 6 7 8 9
<u>5</u> 2 | 6 7 8 9
2 | 5 6 7 8 9

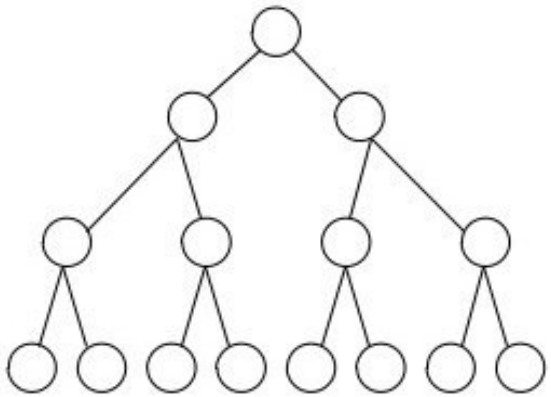
Pseudocode of Heap construction

Stage 1 (heap construction)

```
ALGORITHM  HeapBottomUp(H[1..n])
    //Constructs a heap from elements of a given array
    // by the bottom-up algorithm
    //Input: An array H[1..n] of orderable items
    //Output: A heap H[1..n]
    for i ← ⌊n/2⌋ downto 1 do
        k ← i;   v ← H[k]
        heap ← false
        while not heap and 2 * k ≤ n do
            j ← 2 * k
            if j < n   //there are two children
                if H[j] < H[j + 1]  j ← j + 1
            if v ≥ H[j]
                heap ← true
            else H[k] ← H[j];   k ← j
        H[k] ← v
```

Analysis of Heapsort

Stage 1: Build heap for a given list of *n* keys



Height	Max nodes
h = 3	1
h = 2	2
h = 1	4
h = 0	8

Max nodes at height h = $\left\lceil \frac{n}{2^{h+1}} \right\rceil$

i.e. $h = 0$, Max nodes = $\left\lceil \frac{15}{2^{0+1}} \right\rceil = \left\lceil \frac{15}{2} \right\rceil = \lceil 7.5 \rceil = 8$

c: *constant time for comparison*

$$T(n) = \sum_{h=0}^{\log n} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot ch$$

$$= \sum_{h=0}^{\log n} \left\lceil \frac{n}{2^h \cdot 2} \right\rceil \cdot ch$$

$$= \frac{nc}{2} \sum_{h=0}^{\log n} \left\lceil \frac{h}{2^h} \right\rceil$$

$$= \frac{nc}{2} \sum_{h=0}^{\infty} \left\lceil \frac{h}{2^h} \right\rceil$$

$$= \frac{nc}{2} \cdot 2 = nc$$

$$T(n) \in \Theta(n)$$

Analysis of Heapsort

Stage 2: Repeat operation of root removal n-1 times (fix heap)

$$C(n) \leq 2\lfloor \log_2(n-1) \rfloor + 2\lfloor \log_2(n-2) \rfloor + \cdots + 2\lfloor \log_2 1 \rfloor \leq 2\sum_{i=1}^{n-1} \log_2 i$$

$$\leq 2\sum_{i=1}^{n-1} \log_2(n-1) = 2(n-1)\log_2(n-1) \leq 2n\log_2 n \in \boldsymbol{\Theta(nlogn)}$$

For both stages, $\boldsymbol{\Theta(n) + \Theta(nlogn) = \Theta(nlogn)}$

Comparison of Sort Algorithms

- Comparison of Sort Algorithms

	Number of Comparisons		
	Best	Average	Worst
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Bubble Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Shell Sort	$\Theta(n^{7/6})$	$\Theta(n^{5/4})$	$\Theta(n^2)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Quick Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$

ฟังก์ชัน sort ในภาษา C++

- เป็นการใช้ฟังก์ชัน std::sort จากไลบรารี <algorithm>

ฟังก์ชัน	คำอธิบาย
sort()	เรียงทั้งหมด (เร็วที่สุด)
stable_sort()	เรียงทั้งหมด (รักษาลำดับเดิมของข้อมูลที่มีค่าเท่ากัน)
partial_sort()	เรียงบางส่วน
nth_element()	หาค่าตัวที่ n โดยไม่ต้องเรียงทั้งหมด
is_sorted()	เช็คว่าเรียงลำดับแล้วหรือไม่

1. std::sort (ใน <algorithm>)

- เป็นฟังก์ชันที่ใช้เรียงลำดับอาร์เรย์หรือ vector (Hybrid of QuickSort, HeapSort, and InsertionSort)

```
#include <iostream>
#include <vector>
#include <algorithm> // ใช้ sort()

using namespace std;

int main() {
 vector<int> arr = {5, 2, 9, 1, 5, 6};

 // เรียงจากน้อยไปมาก
 sort(arr.begin(), arr.end());

 // แสดงผลลัพธ์
 for (int num : arr) {
    cout << num << " ";
 }

return 0;
}
```

Syntax of sort()

```
void sort (iterator first, iterator last, Compare comp);
```

comp เป็นฟังก์ชันที่กำหนดลำดับ (เงื่อนไขการเปรียบเทียบ) ว่า element ต่าง ๆ จะถูกจัดเรียงอย่างไร

```
// เรียงจากมากไปน้อย
sort(arr.begin(), arr.end(), greater<int>());
```

Sort Vector of User Defined Type

```
#include <algorithm>
#include <iostream>
#include <vector>
using namespace std;

struct Point{
  int x;
  int y;
};

int main(){
   vector<Point> points = {{3, 5}, {1, 2}, {4, 1}};

// Sort points by y value (ascending)
sort(points.begin(), points.end(), [](const Point &a, const Point &b){ return a.y < b.y; });

for (auto p : points)
  cout << "(" << p.x << "," << p.y << ") ";


return 0;
}
```

(4, 1) (1,2) (3, 5)

2. stable_sort (ใน <algorithm>)

- คล้าย sort แต่รักษาลำดับเดิมของข้อมูลที่มีค่าเท่ากัน (Stable Sorting)

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
  vector<pair<int, char>> arr = {{3, 'A'}, {1, 'B'}, {3, 'C'}, {2, 'D'}};

  stable_sort(arr.begin(), arr.end());

  for (auto p : arr) {
    cout << "(" << p.first << ", " << p.second << ") ";
  }

  return 0;
}
```

(1, B) (2, D) (3,A) (3, C)

3. partial_sort (เรียงบางส่วน)

- ใช้เรียงเฉพาะบางส่วนของอาร์เรย์ เช่น หาตัวที่เล็กที่สุด 3 ตัว

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
  vector<int> arr = {7, 2, 5, 3, 1, 6, 4};

  // เรียง 3 ตัวที่เล็กที่สุดให้อยู่ข้างหน้า
  partial_sort(arr.begin(), arr.begin() + 3,  arr.end());

  for (int num : arr) {
    cout << num << " ";
  }

  return 0;
}
```

```
1 2 3 7 5 6 4
```

4. nth_element (หาค่าลำดับที่ n)

- ใช้หาค่าตัวที่ n โดยไม่ต้องเรียงลำดับทั้งหมด

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
  vector<int> arr = {9, 8, 7, 6, 5, 4, 3, 2, 1, 0};

  // หาตัวที่เล็กที่สุดลำดับที่ 3 (index 2)
  nth_element(arr.begin(), arr.begin() + 2, arr.end());

  cout << "ตัวที่เล็กที่สุดลำดับที่ 3: " << arr[2] << endl;
  cout << "สมาชิกทั้งหมด: ";
  for (int i : arr) {
    cout << i << " ";
  }
  return 0;
}
```

```
ตัวที่เล็กที่สุดลำดับที่ 3: 2
สมาชิกทั้งหมด: 0 1 2 3 4 5 6 7 8 9
```

5. is_sorted (เช็คว่าเรียงลำดับแล้วหรือไม่)

- ใช้เรียงเฉพาะบางส่วนของอาร์เรย์ เช่น หาตัวที่เล็กที่สุด 3 ตัว

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

int main() {
  vector<int> arr = {1, 2, 3, 4, 5};

  if (is_sorted(arr.begin(), arr.end())) {
    cout << "เรียงแล้ว!";
  } else {
    cout << "ยังไม่เรียง!";
  }

  return 0;
}
```

```
เรียงแล้ว!
```

Contest

1. Zero Testing V2
2. Fast Sort
3. 3 Sum
4. Sort Date
5. Check Duplicate
6. Merge Sort (Bottom Up)
7. Queue Reconstruction